

Chapter 4: System Design and Modeling

4.1. Introduction to System Architecture

The structural transition from conceptual requirements to a functional digital infrastructure requires comprehensive architectural modeling to define system behavior accurately. This chapter delineates the foundational blueprints of the Smart Basket Market platform, utilizing standardized modeling languages to illustrate complex interactions across diverse hardware and software boundaries. By isolating the automated self-service checkout processes from traditional retail workflows, the models prioritize decentralized product validation protocols executed directly within the shopping cart module. The core objective of this design is to establish a rigorous, highly available communication mechanism between client applications and central databases without compromising structural security.

Following strict engineering protocols, the subsequent sections partition the systemic framework into multiple isolated perspectives, encompassing functional use cases, chronologically ordered sequence flows, dynamic operational activities, static class structures, and persistent data entities. These multi-dimensional abstractions serve as the technical foundation required to guide the physical deployment of sensor hardware and the programmatic integration of mobile interfaces. Consequently, the depicted models validate the procedural logic responsible for translating physical weight differentials into cryptographic transactional settlements.

4.2. Use Case Model

The Use Case Diagram defines the operational scope of the framework by specifying the authorized actors and their corresponding privileges within the isolated digital ecosystem. This abstraction acts as a boundary definition, strictly outlining which entities may trigger functional processing routines during an active retail session. Within the Smart Basket Market architecture, interactions are predominantly driven by the consumer, while automated physical endpoints execute responsive logic based on verified commands.

4.2.1. Actor Definitions

The system interacts with several distinct entities that dictate operational flow and administrative oversight. The primary defined actors include the following participants:

- Customer: The principal user initiating the self-service shopping lifecycle through digital interactions and mobile payments.
- System Admin: The authorized personnel responsible for overseeing systemic inventory levels and operational dashboard analytics.
- Exit Gate (NodeMCU): The physical hardware endpoint that receives remote trigger commands to automatically manipulate the security barrier mechanism.

4.2.2. Functional Use Cases

The core transactions executed by these defined actors encapsulate the entire autonomous retail journey from inception to physical departure. Each functional use case guarantees strict data validation protocols prior to state transition. The primary operations include:

- Scanning the initialization QR code to logically bind the mobile application to a specific hardware basket.
- Logging physical product barcodes via the tablet scanner interface to fetch verified pricing and mass metadata.
- Validating internal weight differentials automatically using embedded load cell circuits.
- Finalizing the virtual cart summary to generate a cryptographically signed digital invoice code.
- Processing monetary settlement exclusively through the decentralized mobile terminal.
- Verifying the settled transaction at the physical boundary to trigger the solenoid unlock mechanism.

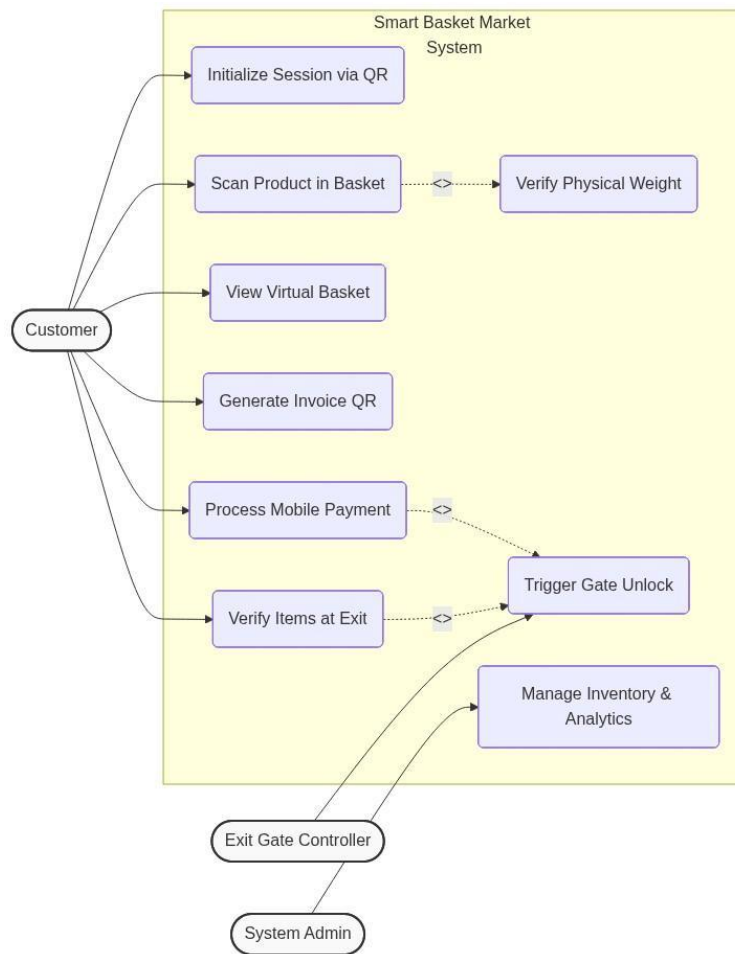


Figure 4.1: System Use Case Diagram

4.3. Sequence Diagram

The Sequence Diagram establishes the exact chronological execution order required to process multi-stage data transactions across distributed hardware nodes. This representation exposes the synchronous and asynchronous communication protocols utilized to maintain data consistency between the active frontend presentation layers and the backend persistence engine. By charting the temporal flow of digital requests, the architecture demonstrates how real-time validation loops mitigate potential structural desynchronization.

During a standard retail interaction, the temporal workflow begins with a session handshake initialized by the mobile client and concludes with a physical hardware actuation command. The intermediate operations operate within a closed processing loop, continuously cross-referencing incoming barcode strings with physical weight measurements captured by the embedded sensors. Only upon the successful culmination of the mobile payment logic does the server dispatch the final clearance signal to the boundary gate microcontroller.

Moreover, the sequence of operations places a significant emphasis on reducing the computational load on the mobile client by delegating heavy validation algorithms to the centralized backend server. The mobile application and the basket tablet function primarily as data capture nodes, responsible for capturing input and rendering server responses. This architectural decision not only enhances the overall security posture by keeping sensitive logic off consumer devices but also ensures that any updates to the pricing engine or weight tolerance algorithms can be deployed instantaneously on the backend without requiring application updates on the client side.

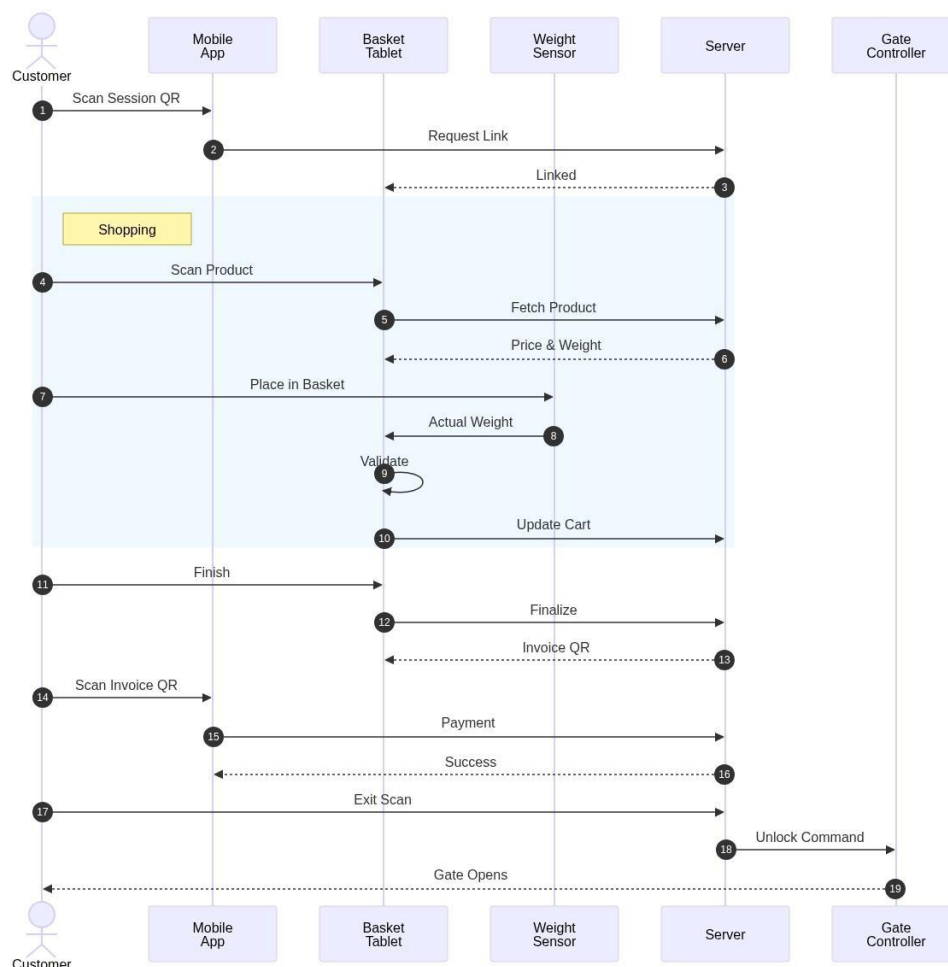


Figure 4.2: System Sequence Diagram

4.4. Activity Diagram

The Activity Diagram maps the dynamic procedural logic and conditional branching structures that govern the automated consumer journey. This behavioral model explicitly outlines the internal decision matrices utilized by the application logic to either advance the transaction or invoke error-handling

subroutines. Every physical action taken by the consumer must satisfy pre-defined systemic constraints before altering the permanent database states.

The procedural logic enforces a rigorous verification loop during the item deposition phase, comparing analog sensor conversions against authorized database metadata. Should the expected mass deviate beyond the acceptable tolerance threshold, the process halts, preventing fraudulent accumulation within the virtual cart. Upon the successful negotiation of the payment portal, the workflow transitions into an automated exit verification sequence, ensuring that physical departure is strictly coupled with successful monetary settlement.

Furthermore, the activity model illustrates the concurrent processing capabilities of the system when handling multiple operations. For instance, while the tablet interface continuously updates the visual cart rendering for the user, the background services maintain an active WebSocket listener awaiting real-time inventory synchronization signals. This approach ensures that the user experience remains highly responsive, even when the system is executing complex cryptographic validations or communicating with external banking APIs for payment authorization. It highlights the fault-tolerant nature of the architecture, where network latency or temporary sensor recalibrations do not result in a complete system crash but instead trigger graceful retry mechanisms.

Finally, the administrative and security implications of the activity flow are clearly demarcated within the final exit stages. The transition from a completed payment state to the physical unlocking of the gate involves multiple verification checkpoints designed to prevent unauthorized egress. The system logs every state change, from the initial scanning of the session QR code to the final solenoid retraction, creating a comprehensive audit trail. This detailed logging capability, embedded within the activity lifecycle, provides system administrators with the necessary telemetry to monitor operational efficiency, detect potential hardware malfunctions, and continuously optimize the autonomous checkout algorithm.

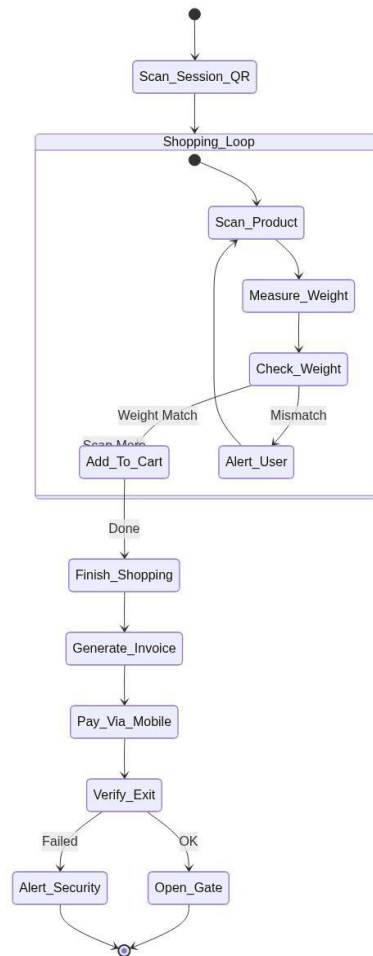


Figure 4.3: System Activity Diagram

4.5. Class Diagram

The Class Diagram provides a static, object-oriented representation of the core software structures governing the platform logic. This architectural view identifies the primary computational entities, their internal attributes, exposing methods, and relational dependencies. By encapsulating functional behavior within distinct class definitions, the framework enforces modularity, ensuring that payment processing, inventory tracking, and session management remain structurally independent.

The relational mapping connects the consumer profile directly to transient shopping sessions, which subsequently bind multiple recorded item objects into a unified digital invoice entity. The integration of the physical weight attribute within the product class definition is a critical dependency required by the automated verification loop. Furthermore, the gate controller class remains completely isolated from the cart logic, interacting solely with finalized payment states to ensure maximum operational security.

In addition to the core structural mapping, the class architecture incorporates critical design patterns that facilitate long-term maintainability and scalability. The separation of concerns is strictly enforced, where the User class handles exclusive authentication logic, while the Session class is responsible for managing the temporal lifecycle of a shopping interaction. This decoupling ensures that any future enhancements to the cryptographic hashing algorithms within the User module do not inadvertently affect the real-time operational flow of active retail sessions.

Furthermore, the CartItem and Product classes demonstrate a robust aggregational relationship that bridges the gap between static inventory data and dynamic transactional context. The Product class serves as an immutable reference blueprint containing baseline attributes such as the theoretical expected weight. Conversely, the CartItem class records the actual empirical data captured by the physical sensors during a specific session. This deliberate separation allows the system to accurately compute deviations and flag potential anomalies without altering the master inventory records, thereby preserving data integrity across all operational phases.

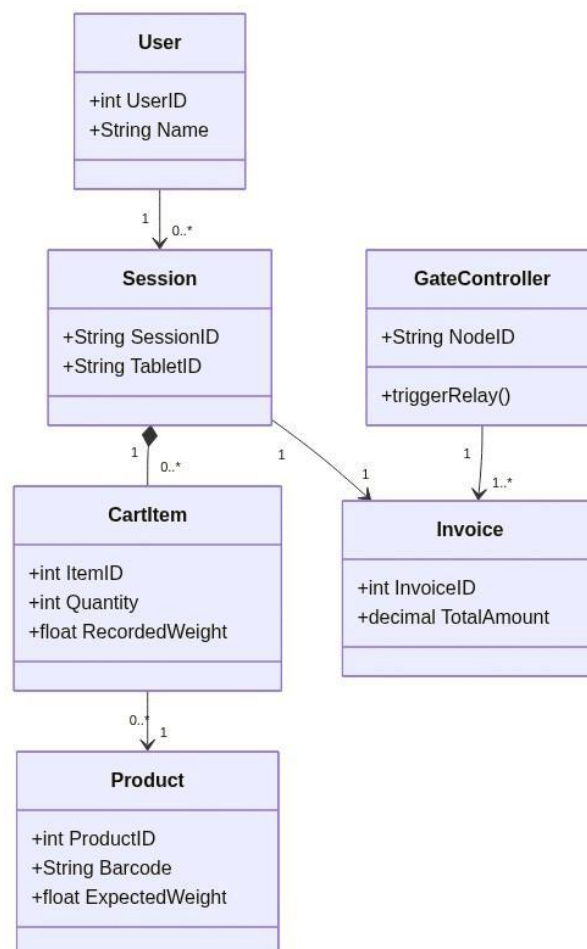


Figure 4.4: System Class Diagram

4.6. Entity Relationship Diagram (ERD)

The Entity Relationship Diagram outlines the relational database schema implemented within the persistent storage layer. This structural blueprint defines the tabular data formations, primary key constraints, and foreign key integrations necessary to guarantee absolute data normalization. Maintaining high relational integrity is paramount to supporting high-velocity concurrent queries originating from multiple distributed basket terminals.

The schema relies on five primary table structures to archive the complex interaction data generated during retail operations. The user table maintains encrypted profile credentials, while the centralized product repository serves as the definitive source for pricing and expected mass variables. All transactional events generate structured records within the sessions, invoices, and invoice items tables, creating an immutable, chronologically traceable ledger suitable for both administrative analytics and system auditing.

To ensure optimal query performance during peak operational hours, the database schema integrates selective indexing strategies across all highly contested foreign keys. The SessionID and UserID fields are heavily indexed to facilitate instantaneous linkage between active consumers and their historical transactional ledgers. This indexing is particularly vital for the mobile application, which must rapidly retrieve previous invoices and payment statuses without experiencing noticeable latency.

Moreover, the schema design enforces strict referential integrity constraints to prevent orphaned records and maintain a cohesive data ecosystem. For instance, the deletion of a User record would trigger a cascaded archival process or restriction rule, ensuring that historical financial data within the Invoices table remains mathematically sound and accessible for auditing purposes. The ERD effectively maps this multi-tiered constraint network, illustrating how the persistence layer guarantees absolute reliability even in the face of unexpected application failures or network interruptions.

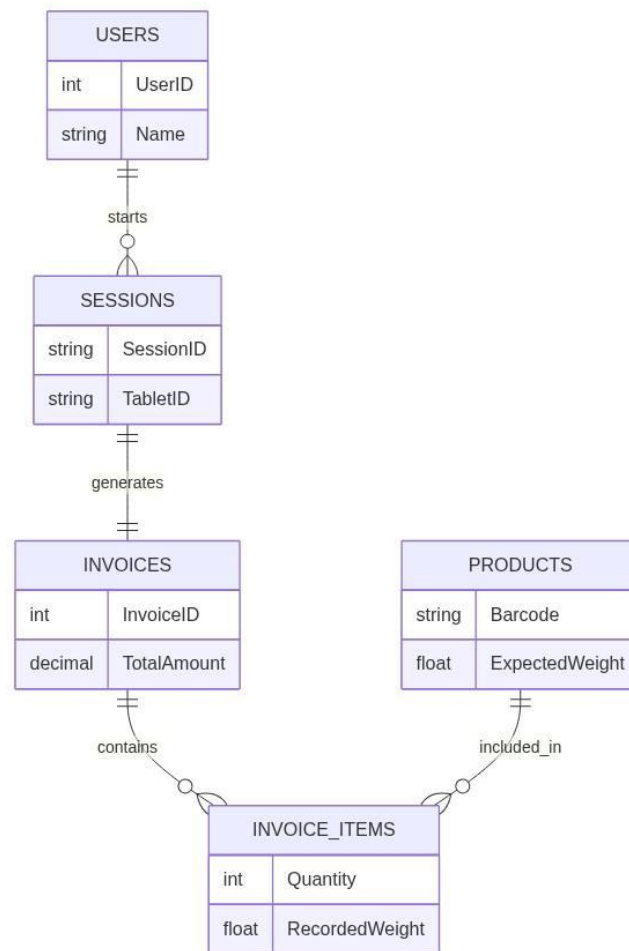


Figure 4.5: Entity Relationship Diagram

4.7. Deployment Diagram

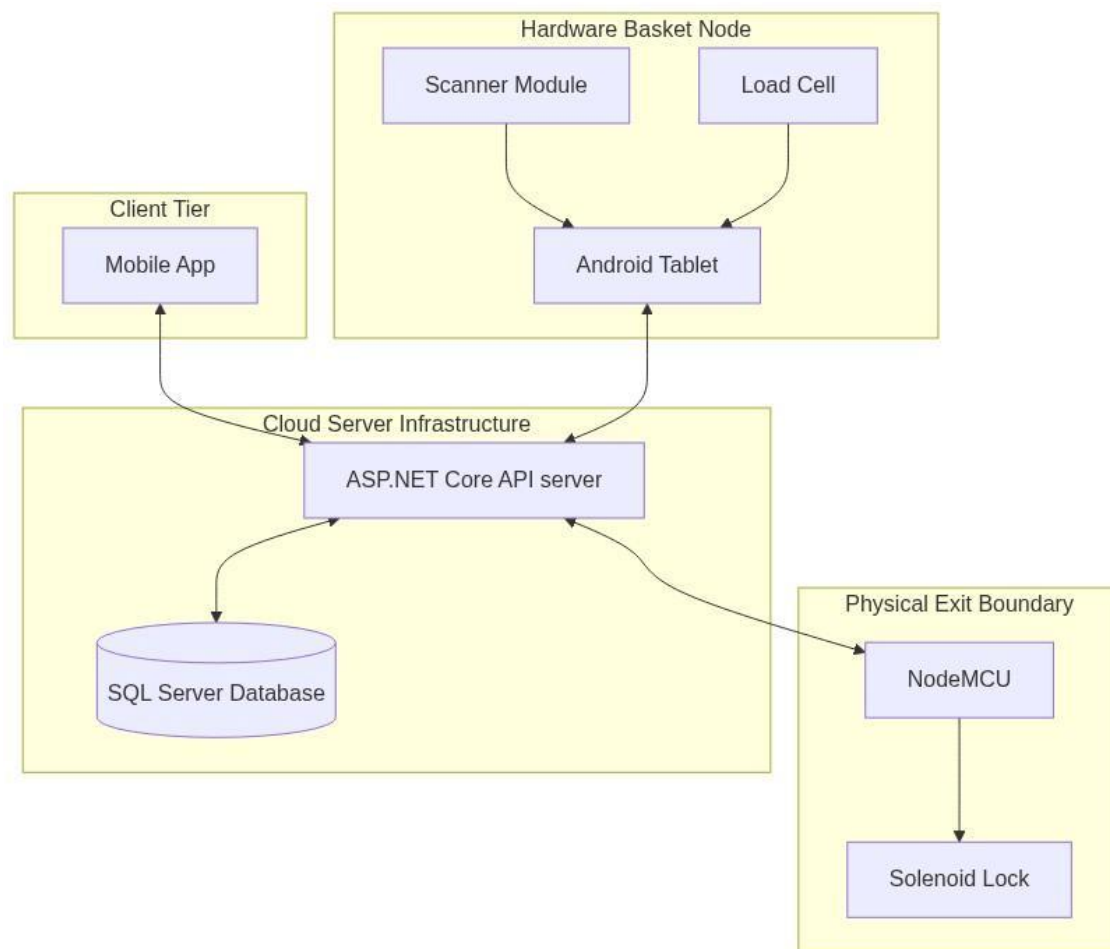


Figure 4.6: System Deployment Diagram

The Deployment Diagram details the physical distribution of hardware computational nodes and the specific communication protocols utilized to network the distributed environment. This topological model illustrates the complex infrastructure required to merge local physical sub-systems with centralized cloud architectures. The physical separation of the mobile processing unit, the basket sensor module, and the exit gate assembly necessitates the implementation of diverse transmission standards.

The hardware configuration leverages wireless HTTPS protocols for standard API consumption, while employing low-latency WebSockets via SignalR for instantaneous gate actuation broadcasts. Locally on the basket assembly, analog-to-digital converters process raw load cell data through universal serial bus interfaces directly into the computing tablet. This multi-layered networking strategy ensures that high-bandwidth operations are offloaded to capable mobile processors, while low-level physical switches receive dedicated, uninterrupted command streams.

Security protocols are inherently integrated into this deployment architecture, ensuring that all data in transit is cryptographically protected against

interception. The communication channels bridging the Client Tier and the Cloud Tier utilize Transport Layer Security (TLS), encrypting sensitive payload information such as payment tokens and user credentials. Meanwhile, the localized hardware interactions between the Tablet and the Microcontroller operate within a closed physical loop, drastically reducing the attack surface against external cyber threats.

Finally, the cloud infrastructure is engineered for high availability and automatic horizontal scaling. The ASP.NET Core API servers are containerized, allowing the orchestration layer to dynamically provision additional computational resources in response to increased consumer traffic. Simultaneously, the SQL Server Database utilizes active geo-replication to safeguard against regional data center failures. This comprehensive deployment strategy guarantees that the Smart Basket Market system can sustain uninterrupted retail operations while providing an extensible foundation for future hardware integrations.